

1. Connection Setup

Cob Staines

2026-08-02

Table of contents

Motivation	1
R	2
Store & access your database connection parameters	2
Access your local .Renvirom file	2
Save connections parameters	2
Establish database connection	3
Load packages	3
Connect	4
Begin using your connection!	4
Disconnect	5
Python	5
Store & access your database connection parameters	6
Create a dbconfig file	6
Establish database connection	6
Import packages	6
Connect	7
Begin using your connection	7
Disconnect	7
DBeaver	8
Set up and test your connection.	8
Disconnect	10

This tutorial is available as a [.qmd on Github](#).

Motivation

- Connect to the RIBBiTR (or another remote) database with ease and repeatability

- Manage login credentials for ease and security, so they won't be lost or shared with your code

R

In this quick tutorial we will (re)orient ourselves to connecting to the RIBBiTR database using R/RStudio. If you aren't yet familiar with R/Rstudio, check out this quick [Getting Started tutorial by POSIT](#).

Specifically, we will be setting up a database connection using the `ribbitrrr::hopToDB()` function (from the `ribbitrrr` R package). If you would prefer to write your own connection script in R, check out the [DBI](#) R-package.

Store & access your database connection parameters

Access your local .Renviron file

Your .Renviron file a local file where you can save and reference your login credentials for easy use within R and RStudio, without risking losing them or potentially sharing them on accident when you share your code. A simple way to access your .Renviron file is with the function `usethis::edit_r_environ()`

```
install.packages("usethis")  
  
# open your local .Reniron file  
usethis::edit_r_environ()
```

Save connections parameters

Copy the following database connection parameters to your .Renviron file, substituting your login credentials (user & password).

```
# RIBBiTR DB credentials  
ribbitr.dbname = "[DATABASE-HERE]"  
ribbitr.host = "[HOST-HERE]"  
ribbitr.port = "[PORT-HERE]"  
ribbitr.user = "[YOUR-USERNAME-HERE]"  
ribbitr.password = "[YOUR-PASSWORD-HERE]"
```

Save and close .Renviron, and restart RStudio.

YAML login credentials

The function `ribbitrrr::hopToDB()` also supports passing a YAML file which contains your login credentials. If using this method, take care not to share this credentials file publicly when sharing code or pushing to github.

Establish database connection

Create a new R project (or .qmd, .Rmd, .R etc.) file where you can follow the tutorial and establish the database connection.

Load packages

Rtools

[Rtools](#) is required to build R packages locally, as part of this tutorial. Check to see if Rtools is installed by running:

```
Sys.which("make")
```

```
      make
"/usr/bin/make"
```

If this returns a path to the make function (e.g. `/usr/bin/make`), Rtools is installed and you can proceed to the next step. If you return an empty string "" you will need to [download Rtools](#) first.

Librarian

“librarian” is a package and library management package in R which makes it easier to install, load, update and unload packages to meet dynamic environment needs. There are other ways to download, load, and maintain packages in R (e.g. `install.packages()` and `library()`), but we recommend librarian for its simplicity and portability.

```
# install and load "librarian" R package
install.packages("librarian")
```

librarian downloads and loads packages using the `librarian::shelf` function. Below are the minimal recommended packages to establish a connection to the RIBBiTR database.

```
# update your ribbitrrr package to the latest version
librarian::shelf(RIBBiTR-BII/ribbitrrr, update_all = TRUE)
```

These packages will be installed:

'ribbitrrr'

It may take some time.

Downloading GitHub repo RIBBiTR-BII/ribbitrrr@HEAD

```
-- R CMD build -----
* checking for file '/tmp/RtmplxW7f0/remotesa0db2851d13a/RIBBiTR-BII-ribbitrrr-8f9be81/DESCRIPTION' ... OK
* preparing 'ribbitrrr':
* checking DESCRIPTION meta-information ... OK
* checking for LF line-endings in source and make files and shell scripts
* checking for empty or unneeded directories
* building 'ribbitrrr_0.0.2.4.tar.gz'
```

```
# minimal packages for establishing RIBBiTR DB connection
librarian::shelf(tidyverse, dbplyr, RPostgres, DBI, RIBBiTR-BII/ribbitrrr)
```

Connect

Now, using the `ribbitrrr:hopToDB()` function, let's establish a connection!

```
# establish database connection
dbcon <- hopToDB("ribbitr")
```

Connecting to 'ribbitr'... Success!

`hopToDB()` returns a database connection object (`dbcon`). Keep track of this connection object, we will call it to explore and pull data later.

Begin using your connection!

Try out your connection by loading table metadata from the database:

```
mdt <- tbl(dbcon, Id("public", "all_tables")) %>%
  collect()
head(mdt)
```

```
# A tibble: 6 x 4
  table_schema      table_name      column_count table_description
  <chr>             <chr>             <int64> <chr>
1 microclimate_data logger                    5 "data logger lookup table (on~
2 microclimate_data metadata_columns          25 "Column-specific definitions ~
3 microclimate_data metadata_tables           4 "Table-specific definitions a~
4 microclimate_data sensor                    5 "microclimate sensor lookup t~
5 microclimate_data ts_dew_point              3 "Dew point time series data t~
6 microclimate_data ts_illuminance            3 "Illuminance time series data~
```

💡 Connection browsing in RStudio

By default the `ribbitrrr::hopToDB()` function displays a browsable database interface in the **Connections** panel upon establishing a new connection. In this panel you can view all accessible table, column names and data types, as well as preview the data for context. Try it out!

Disconnect

It is good practice to close your database connection, to let the server know it can stop listening for you (otherwise it will continue to use server resources in anticipation of your). You can think of this as saying goodbye at the end of a phone call.

```
# disconnect from database
dbDisconnect(dbcon)
```

💡 Also try...

- For those managing multiple database connections, the `hopToDB()` function allows you to store and fetch various sets of login credentials with a single keyword. Just substitute “ribbitr” in the `.Renviron` example above with your own keywords to juggle multiple logins.
- Login credentials saved to `.Renviron` can also be accessed explicitly anytime using `Sys.getenv("ribbitr.dbname")`, etc. In most cases the `ribbitrrr::hopToDB()` function is all you will need.

Python

Here is a quick tutorial to (re)orient ourselves to connecting to the RIBBiTR database using Python. If you aren't yet familiar with Python, check out this quick [Getting Started tutorial](#) by DATAQUEST.

Store & access your database connection parameters

Create a dbconfig file

We recommend you create a local database config (`dbconfig.py`) file where you can save and reference your login credentials for easy use in python, without risking losing them or potentially sharing them on accident when you share your code.

Create a file named `dbconfig.py` in your project working directory (or another preferred location, see “Also try” below). Copy the following to `dbconfig.py`:

```
# dbconfig.py

ribbitr = {
    "database": "[DATABASE-HERE] ",
    "host": "[HOST-HERE] ",
    "port": "[PORT-HERE] ",
    "user": "[YOUR-USERNAME-HERE] ",
    "password": "[YOUR-PASSWORD-HERE] ",
}
```

Save `dbconfig.py`.

Note

Be sure to add `dbconfig.py` to your local `.gitignore` file if you are using git/github, so you don't accidentally publish your login credentials!

Establish database connection

Create a new `.py` (or `.qmd`, `.ipynb`, etc.) file where you can follow the tutorial and establish the database connection.

Import packages

This method requires installing the `ibis.postgres` package to your working environment, in addition to `pandas`. We also import the `dbconfig.py` file to access your login credentials.

```
import ibis
import pandas as pd
import dbconfig # import connection credentials
```

Connect

Now, using the `ibis.postgres.connect()` function, let's establish a connection!

```
# establish database connection
dbcon = ibis.postgres.connect(**dbconfig.ribbitr)
```

`ibis.postgres.connect()` returns a database connection object (`dbcon`). Keep track of this connection object, we will call it to explore and pull data later.

Begin using your connection

Try out your connection by loading table metadata from the database

```
mdt = dbcon.table(database = "public", name = "all_tables").to_pandas()
mdt.head()
```

	table_schema	...	table_description
0	microclimate_data	...	data logger lookup table (one logger contains ...
1	microclimate_data	...	Column-specific definitions and metadata ("dat...
2	microclimate_data	...	Table-specific definitions and metadata ("data...
3	microclimate_data	...	microclimate sensor lookup table
4	microclimate_data	...	Dew point time series data table

[5 rows x 4 columns]

Disconnect

It is good practice to close your database connection, to let the server know it can stop listening for you (otherwise it will continue to use server resources in anticipation of your). You can think of this as saying goodbye at the end of a phone call.

```
# close the connection
dbcon.disconnect()
```

💡 Also try...

- For those managing multiple database connections, this method allows you to store and fetch various sets of login credentials with a single keyword. Just substitute “ribbitr” in the `dbconfig.py` file with your own keywords and call them as needed!
- If you will be connecting to the database from different python projects, you

may want to save your `dbconfig.py` file to a more general location. In this case, include the following lines in each of your project files:

```
import sys
sys.path.append("/path/to/dbconfig/dir/")
import dbconfig
```

DBeaver

In this tutorial we will (re)orient ourselves to connecting to the RIBBiTR database using DBeaver. DBeaver is a free, universal database tool which provides a great visual platform and robust backend for interacting with databases. [Download DBeaver](#) and install to get started with the tutorial.

Set up and test your connection.

- 1) Open DBeaver.
- 2) Right-click in the Database Navigator” panel and select **Create -> Connection**

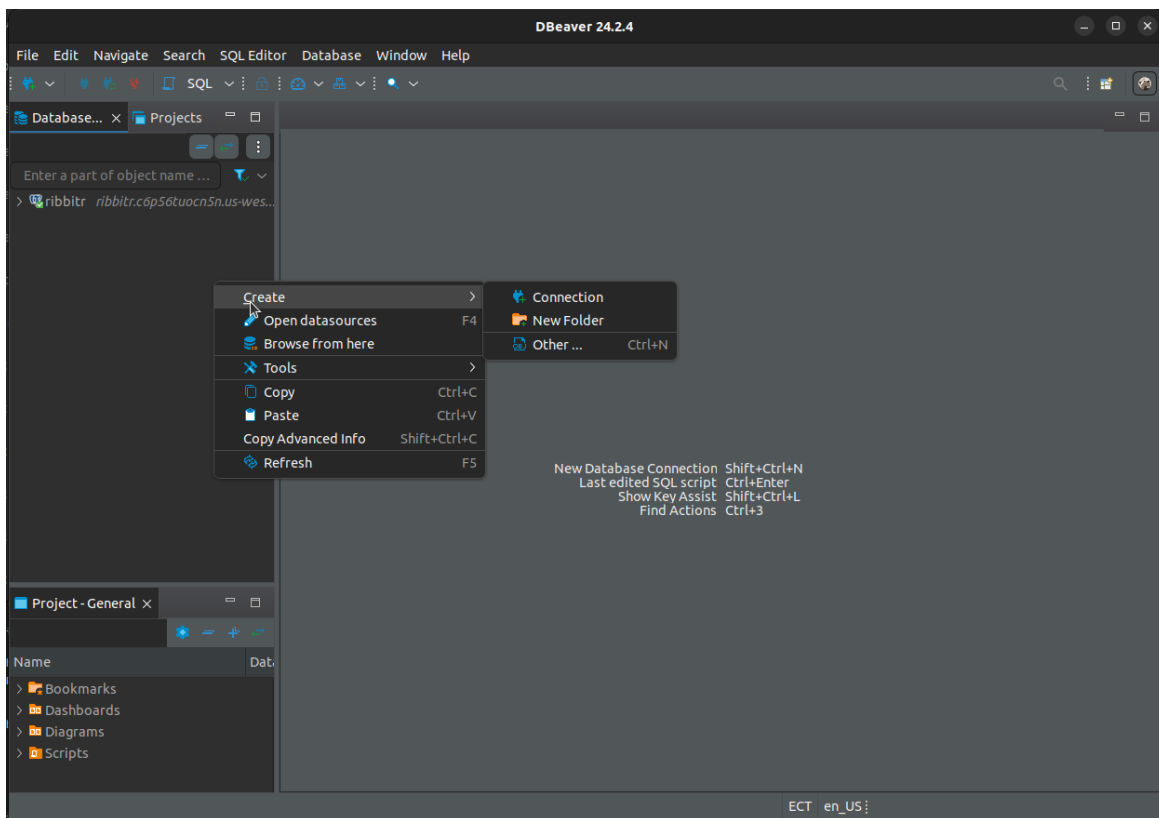


Figure 1: Right-click in the Database Navigator” panel and select **Create -> Connection**

3) Enter your database credentials as shown above, substituting your user and password.

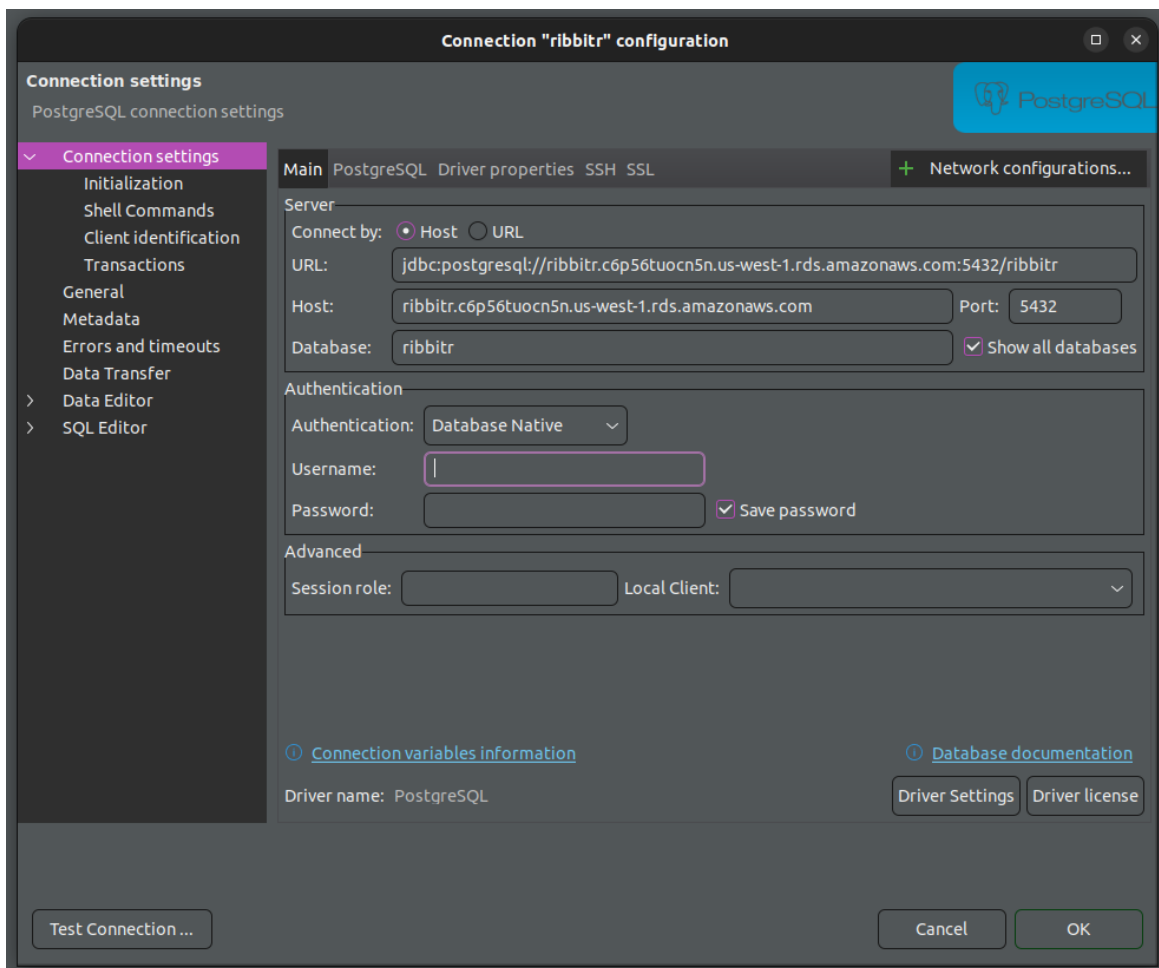


Figure 2: Enter your database credentials as shown above, substituting the database name, host, port, your user and password.

- 4) Click **Test Connection** in the bottom left to make sure your credentials work. Then click **OK**.

That's it, you are ready to start using your connection!

Disconnect

When you are done, click the red **Disconnect** button in top left corner to terminate your connection.

It is good practice to close your database connection, to let the server know it can stop listening for you (otherwise it will continue to use server resources in anticipation of your). You can think of this as saying goodbye at the end of a phone call.

2. Data Discovery ->